EE596 – Video and Image Coding

Mini project - 12th week progress

3.1 Stage 1: Basic implementation: Image compression

1. Codec Architecture

The implemented system serves as the foundational spatial compression module for a hybrid video codec. To maximize compression efficiency, the input RGB image is first converted into the YCbCr color space. This allows the system to exploit the human visual system's varying sensitivities by preserving structural luminance (Y) detail while aggressively compressing chrominance (Cb, Cr) data. The image channels are padded to dimensions divisible by 8 and processed in discrete 8x8 macroblocks to prepare for frequency-domain transformation.

2. Block Diagram Implementation

The system architecture is strictly modularized to reflect the standard block based image coding pipeline.

- Forward Transform:
  Each 8x8 block undergoes a 2D Discrete Cosine Transform (DCT) to convert spatial pixel data into frequency coefficients. A DC offset is applied prior to transformation by subtracting 128 to center the signal energy.
- Quantization:
  This lossy stage scales down the DCT coefficients by dividing them by a quantization matrix and rounding to the nearest integer. Distinct standard matrices are used for the Luma and Chroma channels. A global scaling factor Q_{scale} is applied here to dictate the overall quality and bitrate.
- Entropy Encoding:
  The quantized coefficients, which now contain a high density of zeros, are losslessly compressed into a binary stream. In this implementation, the zlib library simulates the Run-Length and Huffman coding processes to yield the final compressed bit size.
- Decoder Pathway (Inverse Operations):
  The simulated receiver unpacks the bitstream (Entropy Decoding), multiplies by the quantization matrix (Reconstruction of Quantized Data), and applies the

Inverse 2D-DCT to perfectly reconstruct the altered spatial pixels before merging the channels back to RGB

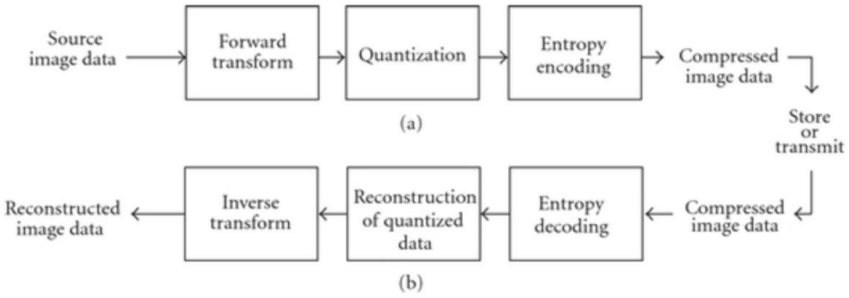


Figure 1: basic image compression system

```python
# ENCODER

# Subtracts DC offset and applies 2D-DCT.
def forward_transform(block):
    return dct2(block - 128)

# Divides by the Quantization matrix and rounds (The Lossy Step).
def quantization(dct_block, Q_matrix):
    return np.round(dct_block / Q_matrix)

# Losslessly compresses the quantized data into a binary stream.
def entropy_encoding(quantized_blocks):
    quantized_array = np.array(quantized_blocks, dtype=np.int16)
    compressed_bytes = zlib.compress(quantized_array.tobytes())
    return compressed_bytes, len(compressed_bytes) * 8, quantized_array.shape
```

```python
# DECODER

# Decompresses the binary stream back into block matrices.
def entropy_decoding(compressed_bytes, original_shape):
    decompressed_bytes = zlib.decompress(compressed_bytes)
    quantized_array = np.frombuffer(decompressed_bytes, dtype=np.int16)
    return list(quantized_array.reshape(original_shape))

# Multiplies by the Q-matrix to restore approximate frequency values.
def reconstruction_of_quantized_data(quantized_block, Q_matrix):
    return quantized_block * Q_matrix

# Applies Inverse 2D-DCT and adds back the DC offset.
def inverse_transform(dequantized_block):
    return idct2(dequantized_block) + 128
```

Figure 2 : Basic building blocks of the image compression system

3. Rate Control Algorithm Design

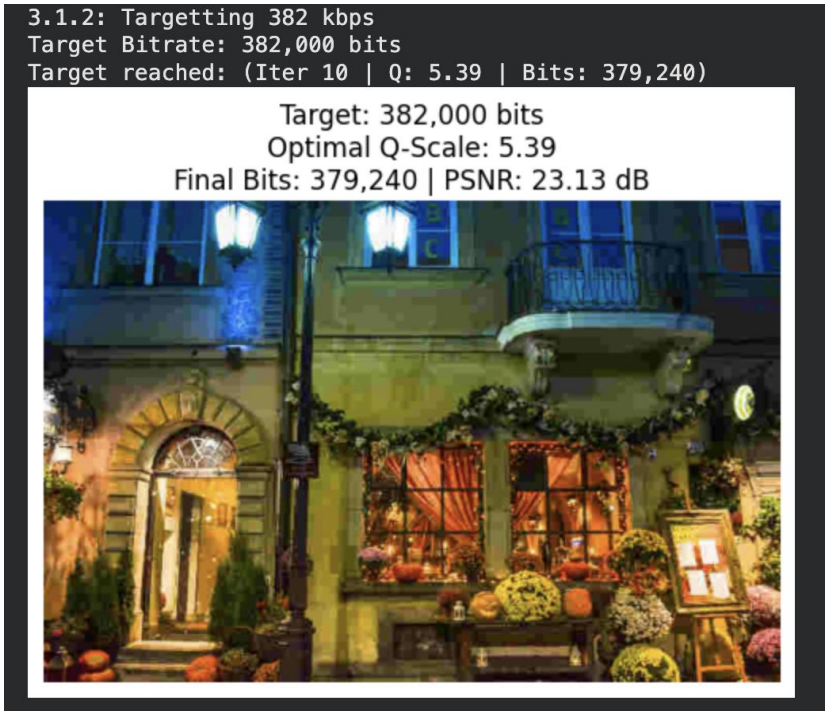
To ensure the output can adapt to strict channel bandwidth constraints specifically the calculated target of 382 kbps for this project a Rate Distortion optimization loop was implemented using a Binary Search algorithm. The algorithm acts as a feedback loop that dynamically adjusts the quantization scaling factor Q_{scale}.

- It establishes a minimum and maximum boundary for Q_{scale} and encodes the image using the midpoint.
- If the resulting compressed bit size exceeds the target bitrate, the algorithm raises the lower boundary to force higher compression.
- If the bit size is below the target, it lowers the upper boundary to reduce compression and improve the Peak Signal-to-Noise Ratio (PSNR).
- This loop repeats until the final bit count converges within a strict 2% error tolerance of the requested bandwidth, guaranteeing the best possible PSNR for any arbitrary bitrate constraint.

4. Results

3.1.1) Implement a basic image coding system which comprise of the block diagrams show in Fig 1, with 3 level of quantisation values. Define the quantisation values to get low, medium and high quality outputs.

3.1.2) Adjust the compression ratio of your image encoder to meet the following bit rates. Assume that you will be transmitting one image per second and the output should be at the best possible PSNR value.



3.1.3) Implement an algorithm where output of your image compression can adopt to any given bitrate.

```
3.1.3: Adapting to any bitrate

Testing Target: 150 kbps...
Target Bitrate: 150,000 bits
Target reached: (Iter 5 | Q: 16.59 | Bits: 152,056)

Testing Target: 300 kbps...
Target Bitrate: 300,000 bits
Target reached: (Iter 10 | Q: 7.33 | Bits: 299,680)

Testing Target: 600 kbps...
Target Bitrate: 600,000 bits
Target reached: (Iter 12 | Q: 2.83 | Bits: 599,792)
```

REFERENCES

1. M. Virtuoso, "JPEG-Image-Compressor: A Python program that compresses raw images based on the JPEG compression algorithm," GitHub repository, 2023. [Online]. Available: https://github.com/mVirtuoso21/JPEG-Image-Compressor
2. S. Benitozzi, "Grayscale image compression using the Bidimensional Discrete Cosine Transform (DCT)," GitHub repository, 2023. [Online]. Available: https://github.com/simonebenitozzi/jpeg-compression